

กรณีศึกษาที่ 4: ระบบจัดอันดับคะแนนสอบ

1. โจทย์ปัญหา

อาจารย์ต้องการเรียงคะแนนสอบของนักศึกษาจากมากไปน้อย โดยใช้เมธอดสำเร็จรูปของภาษา Java เช่น Arrays.sort() ตัวอย่างข้อมูล เช่น

```
int[] scores = {72, 91, 65, 88, 79};
```

2. ข้อมูลนำเข้าและผลลัพธ์ที่ต้องการ

รายการ	รายละเอียด
ข้อมูลนำเข้า (Input)	อาร์เรย์คะแนนสอบ int[] scores ที่ยังไม่ได้เรียงลำดับ เช่น {72, 91, 65, 88, 79}
ผลลัพธ์ (Output)	อาร์เรย์คะแนนชุดเดิมที่ถูกเรียงจากมากไปน้อย เช่น {91, 88, 79, 72, 65} พร้อมจำนวนครั้งของการเปรียบเทียบและการสลับข้อมูล

3. ความหมายของขนาดข้อมูล (n)

กำหนดให้ n คือจำนวนคะแนนสอบทั้งหมดในอาร์เรย์ scores ($n = \text{scores.length}$)

ซึ่งเป็นตัวแปรหลักที่กำหนดจำนวนรอบการเปรียบเทียบและการสลับข้อมูลของอัลกอริทึม

4. แนวคิดและ Pseudocode

ออกแบบด้วย Bubble Sort โดยเปรียบเทียบข้อมูลที่อยู่ติดกันทีละคู่ หากคู่ใดเรียงลำดับผิด (ค่าน้อยอยู่ก่อนค่ามาก) ให้สลับตำแหน่งกัน ทำซ้ำจนครบ n-1 รอบ เพื่อให้ค่ามากที่สุด "ลอย" ไปอยู่ด้านหน้าที่ละตัวในแต่ละรอบ

```
BUBBLE_SORT(scores, n)
  comparisons = 0
  swaps = 0
  for i = 0 to n-2
    for j = 0 to n-2-i
      comparisons = comparisons + 1
      if scores[j] < scores[j+1]
        swap(scores[j], scores[j+1])
        swaps = swaps + 1
  return scores, comparisons, swaps
```

5. โปรแกรมภาษา Java

เมธอดหลักสำหรับเรียงคะแนนจากมากไปน้อยด้วย Bubble Sort แบบพื้นฐาน พร้อมนับจำนวนครั้งของการเปรียบเทียบและการสลับข้อมูล:

```
import java.util.Arrays;

public class ScoreRanking {
```

```
// เรียงคะแนนจากมากไปน้อยด้วย Bubble Sort (แบบพื้นฐาน)
static void bubbleSort(int[] scores) {
    int n = scores.length;
    int comparisons = 0;
    int swaps = 0;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            comparisons++;
            if (scores[j] < scores[j + 1]) {
                int temp = scores[j];
                scores[j] = scores[j + 1];
                scores[j + 1] = temp;
                swaps++;
            }
        }
    }

    System.out.println("จำนวนครั้งที่เปรียบเทียบ: " + comparisons);
    System.out.println("จำนวนครั้งที่สลับข้อมูล: " + swaps);
}

public static void main(String[] args) {
    int[] scores = {72, 91, 65, 88, 79};
    bubbleSort(scores);
    System.out.println(Arrays.toString(scores));
}
}
```

6. Basic Operation ที่ใช้วิเคราะห์

คำสั่งหลัก (Basic Operation) ที่ใช้วิเคราะห์คือ การเปรียบเทียบค่า `scores[j]` กับ `scores[j+1]` (บรรทัด `comparisons++` / `if`) เนื่องจากเป็นคำสั่งที่ทำงานในทุกรอบของลูปชั้นในอย่างแน่นอน ไม่ว่าข้อมูลจะอยู่ในสภาพใดก็ตาม ส่วนการสลับข้อมูล (`swap`) เป็นคำสั่งที่ทำงานแบบมีเงื่อนไข ขึ้นอยู่กับลักษณะของข้อมูล

7. วิเคราะห์จำนวนครั้งที่คำสั่งหลักทำงาน

ลูปด้านนอก `i` วิ่งตั้งแต่ 0 ถึง `n-2` (รวม `n-1` รอบ) และในแต่ละรอบที่ `i` ลูปด้านใน `j` จะทำงาน `(n-1-i)` ครั้ง ดังนั้นจำนวนครั้งของการเปรียบเทียบทั้งหมดคือ:

$$\begin{aligned}\text{จำนวนการเปรียบเทียบ} &= (n-1) + (n-2) + \dots + 1 \\ &= n(n-1) / 2 \\ &= O(n^2)\end{aligned}$$

จำนวนครั้งนี้คงที่เสมอ ไม่ว่าข้อมูลจะเรียงอยู่แล้วหรือไม่ เพราะ Bubble Sort แบบพื้นฐานไม่มีการตรวจสอบเพื่อหยุดก่อนกำหนด ส่วนจำนวนครั้งของการสลับข้อมูล (`swaps`) จะแปรผันตามความไม่เรียงลำดับของข้อมูล อยู่ระหว่าง 0 ถึง $n(n-1)/2$ ครั้ง

8. วิเคราะห์ Best Case, Worst Case และ Average Case

พิจารณาจากคำสั่งหลัก (การเปรียบเทียบ) ของ Bubble Sort แบบพื้นฐาน:

- Best Case: ข้อมูลเรียงจากมากไปน้อยอยู่แล้ว (ตรงกับผลลัพธ์ที่ต้องการ) — ยังคงต้องเปรียบเทียบครบ $n(n-1)/2$ ครั้งเช่นเดิม เพียงแต่ไม่มีการสลับข้อมูลเลย (swaps = 0)
- Worst Case: ข้อมูลเรียงย้อนกลับ (น้อยไปมาก) — ต้องเปรียบเทียบและสลับข้อมูลครบทุกคู่ที่เป็นไปได้ คือ $n(n-1)/2$ ครั้งทั้งการเปรียบเทียบและการสลับ
- Average Case: โดยเฉลี่ยแล้วจะมีการสลับข้อมูลประมาณครึ่งหนึ่งของจำนวนคู่ทั้งหมด แต่จำนวนการเปรียบเทียบยังคงเป็น $n(n-1)/2$ ครั้งเท่าเดิม

จะเห็นว่าเมื่อพิจารณาจากคำสั่งหลัก (การเปรียบเทียบ) แล้ว Bubble Sort แบบพื้นฐานมีจำนวนครั้งการทำงานเท่ากันทั้งสามกรณี คือ $n(n-1)/2$ ครั้งเสมอ

9. Time Complexity

สัญกรณ์	ค่า
Big O (ขอบเขตบน)	$O(n^2)$
Omega (ขอบเขตล่าง)	$\Omega(n^2)$
Theta (ขอบเขตแน่นอน)	$\Theta(n^2)$

เนื่องจาก Best Case และ Worst Case ของ Bubble Sort แบบพื้นฐาน (เมื่อพิจารณาจากจำนวนการเปรียบเทียบ) มีค่าเท่ากันคือ $n(n-1)/2$ จึงสามารถสรุปเป็น $\Theta(n^2)$ ได้ทั้งสามสัญกรณ์

10. Auxiliary Space Complexity

อัลกอริทึมเป็นการเรียงลำดับแบบ in-place ใช้ตัวแปรเพิ่มเติมเพียงไม่กี่ตัว (temp, comparisons, swaps, ตัวแปร loop i และ j) ซึ่งมีขนาดคงที่ ไม่ขึ้นกับ n ดังนั้น Auxiliary Space Complexity = $O(1)$

11. การวิเคราะห์กรณีข้อมูลเรียงแล้ว และเรียงย้อนกลับ

11.1 กรณีข้อมูลเรียงลำดับไว้แล้ว (มากไปน้อย)

Bubble Sort แบบพื้นฐานยังคงวนลูปครบทุกรอบและเปรียบเทียบครบ $n(n-1)/2$ ครั้งเหมือนเดิม เนื่องจากไม่มีกลไกตรวจสอบว่าข้อมูลเรียงลำดับแล้วหรือยัง แต่จำนวนการสลับข้อมูล (swaps) จะเท่ากับ 0 เพราะไม่มีคู่ใดที่เรียงผิดลำดับเลย

11.2 กรณีข้อมูลเรียงย้อนกลับ (น้อยไปมาก)

เป็นกรณีที่เลวร้ายที่สุด (Worst Case) ทุกคู่ที่ถูกเปรียบเทียบจะต้องสลับตำแหน่งกันเสมอ ทำให้จำนวนครั้งของการเปรียบเทียบและการสลับข้อมูลเท่ากัน คือ $n(n-1)/2$ ครั้งทั้งคู่

12. งานเพิ่มเติม: ปรับปรุง Bubble Sort ด้วยตัวแปร swapped

เพิ่มตัวแปร boolean swapped ในแต่ละรอบของลูปด้านนอก หากผ่านลูปด้านในครบหนึ่งรอบแล้วไม่มีการสลับข้อมูลเลย แสดงว่าข้อมูลเรียงลำดับสมบูรณ์แล้ว จึงสามารถหยุดการทำงานได้ทันทีโดยไม่ต้องวนลูปที่เหลือ

```
static void bubbleSortImproved(int[] scores) {
    int n = scores.length;
    int comparisons = 0;
```

```

int swaps = 0;

for (int i = 0; i < n - 1; i++) {
    boolean swapped = false;

    for (int j = 0; j < n - 1 - i; j++) {
        comparisons++;
        if (scores[j] < scores[j + 1]) {
            int temp = scores[j];
            scores[j] = scores[j + 1];
            scores[j + 1] = temp;
            swaps++;
            swapped = true;
        }
    }
}

// หากรอบนี้ไม่มีการสลับข้อมูลเลย แสดงว่าข้อมูลเรียงลำดับแล้ว จึงหยุดได้ทันที
if (!swapped) {
    break;
}

System.out.println("จำนวนครั้งที่เปรียบเทียบ: " + comparisons);
System.out.println("จำนวนครั้งที่สลับข้อมูล: " + swaps);
}

```

13. เปรียบเทียบ Best Case / Worst Case ระหว่างสองวิธี

รายการ	Bubble Sort ปกติ	Bubble Sort ปรับปรุง	หมายเหตุ
Best Case (ข้อมูลเรียงแล้ว)	$O(n^2)$ — เปรียบเทียบครบ $n(n-1)/2$	$O(n)$ — เปรียบเทียบเพียง $n-1$ ครั้งแล้วหยุด	ปรับปรุงเร็วกว่ามากเมื่อข้อมูลเกือบเรียงแล้ว
Worst Case (ข้อมูลเรียงย้อนกลับ)	$O(n^2)$	$O(n^2)$	ทั้งสองวิธีเท่ากัน เพราะทุกรอบมีการสลับจนถึงรอบสุดท้าย
Auxiliary Space	$O(1)$	$O(1)$	ใช้พื้นที่เพิ่มคงที่เท่ากัน

14. คำถามวิเคราะห์

14.1 Bubble Sort แบบปกติกับแบบปรับปรุงมี Best Case ต่างกันอย่างไร

แบบปกติมี Best Case เป็น $O(n^2)$ เสมอ เพราะไม่มีการตรวจสอบว่าข้อมูลเรียงลำดับแล้วหรือยัง
 จึงวนลูปครบทุกรอบไม่ว่าข้อมูลจะเรียงไว้แล้วหรือไม่ ส่วนแบบปรับปรุงมี Best Case เป็น $O(n)$
 เนื่องจากเมื่อผ่านรอบแรกแล้วไม่พบการสลับข้อมูลเลย (swapped = false) โปรแกรมจะหยุดทำงานทันที
 ทำให้ในกรณีที่ข้อมูลเรียงลำดับไว้แล้วใช้การเปรียบเทียบเพียง $n-1$ ครั้งเท่านั้น

14.2 Worst Case ของทั้งสองวิธีต่างกันหรือไม่

ไม่ต่างกัน ทั้งสองวิธีมี Worst Case เป็น $O(n^2)$ เท่ากัน เนื่องจากในกรณีข้อมูลเรียงย้อนกลับสมบูรณ์
ทุกรอบของลูปด้านนอกจะมีการสลับข้อมูลเกิดขึ้นเสมอ (swapped = true ตลอด) ตัวแปร swapped
ของเวอร์ชันปรับปรุงจึงไม่มีโอกาสหยุดลูปก่อนกำหนด ทำให้ต้องทำงานจนครบทุกรอบเช่นเดียวกับแบบปกติ

14.3 หากมีข้อมูลจำนวน 100,000 ค่า ควรเลือกวิธีนี้หรือไม่

ไม่ควร เนื่องจาก Bubble Sort ทั้งสองแบบมี Time Complexity ในกรณีทั่วไปและกรณีที่เลวร้ายที่สุดเป็น $O(n^2)$ เมื่อ $n = 100,000$
จะต้องเปรียบเทียบข้อมูลสูงสุดถึงประมาณ $n(n-1)/2 \approx 4,999,950,000$ ครั้ง ซึ่งใช้เวลาประมวลผลนานเกินกว่าจะยอมรับได้ในทางปฏิบัติ
ควรเลือกใช้อัลกอริทึมการเรียงลำดับที่มีประสิทธิภาพสูงกว่า เช่น Merge Sort หรือ Quick Sort ที่มี Time Complexity เฉลี่ยเพียง $O(n \log n)$ หรือใช้เมธอด Arrays.sort() ของ Java ซึ่งได้รับการปรับแต่งประสิทธิภาพไว้แล้ว

15. แนวทางปรับปรุงอัลกอริทึมให้มีประสิทธิภาพมากขึ้น

- เปลี่ยนไปใช้อัลกอริทึมที่มี Time Complexity ต่ำกว่า เช่น Merge Sort หรือ Quick Sort ($O(n \log n)$) แทน Bubble Sort เมื่อข้อมูลมีขนาดใหญ่
- หากข้อมูลมีแนวโน้มเกือบเรียงลำดับอยู่แล้ว (nearly sorted) การใช้ Bubble Sort แบบปรับปรุง (มี swapped flag) หรือ Insertion Sort จะมีประสิทธิภาพดีกว่า เพราะสามารถหยุดก่อนกำหนดได้
- ใช้เมธอดมาตรฐานของภาษา เช่น Arrays.sort() ซึ่งใช้อัลกอริทึมแบบผสม (Dual-Pivot Quicksort สำหรับข้อมูลชนิดพื้นฐาน) ที่ผ่านการปรับแต่งประสิทธิภาพมาแล้ว หากไม่มีข้อกำหนดให้เขียนอัลกอริทึมเอง